

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



≡ Capabilities > Persistence

Capabilities

Persistence

📄 Copy page



Persistence lets LangGraph applications keep useful information beyond a single graph run. It matters when an agent needs to continue a conversation, resume after an interruption, recover from a failure, or remember information across interactions.

LangGraph provides two complementary persistence systems:

- **Checkpointers** persist a thread's graph state as checkpoints. Use them for short-term, thread-scoped memory, including conversation continuity, human-in-the-loop workflows, time travel, and fault tolerance.
- **Stores** persist application-defined data outside the graph state. Use them for long-term, cross-thread memory, including user preferences, facts, and shared knowledge.

Most applications can use both: a checkpointer tracks the current thread, and a store tracks durable information across threads.

Quickstart

Compile your graph with a checkpointer, a store, or both:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
graph = builder.compile(checkpointer=checkpointer, store=store)

result = graph.invoke(
    {"messages": [{"role": "user", "content": "Hi, my name is Bob."}]},
    {"configurable": {"thread_id": "thread-1"}},
)
```

i Agent Server handles persistence automatically When using the [Agent Server](#), you do not need to implement or configure checkpointer or stores manually. The server handles persistence infrastructure behind the scenes.

Checkpoint vs. store

	Checkpoint	Store
Persists	Graph state snapshots	Application-defined key-value data
Scope	A single thread	Across threads
Memory type	Short-term, thread-scoped memory	Long-term, cross-thread memory
Use for	Conversation continuity, human-in-the-loop, time travel, and fault tolerance	User preferences, facts, and shared knowledge
Access pattern	Pass a <code>thread_id</code> in graph config	Read and write items from nodes or application code
Full guide	Checkpoints	Stores

Troubleshooting common issues

PostgresSaver: thread_id too long

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
import uuid

config = {"configurable": {"thread_id": str(uuid.uuid4())[:255]}}
```

MemorySaver does not persist between restarts

`MemorySaver` and `InMemorySaver` store checkpoints in RAM. When the process restarts, all checkpoints are lost.

Fix: Use a persistent checkpointer for production:

- `PostgresSaver` : PostgreSQL with async support
- `SqliteSaver` : Local file-based storage for development

Checkpoints growing unboundedly

Over long conversations, checkpoints accumulate. This can increase latency and storage costs.

Fix: Prune old checkpoints periodically or set a retention policy:

```
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string("postgresql://...")
checkpointer.setup() # Creates tables with indexes
# Consider adding a cron job to delete checkpoints older than N days
```

State access from parent graph to subgraph

When a subgraph updates state, the parent graph may not see the changes immediately. This is because each subgraph manages its own checkpoint namespace.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



- [Use stores](#) to persist durable data across threads.

 [Connect these docs](#) to Claude, VSCode, and more via MCP for real-time answers.

 [Edit this page on GitHub](#) or [file an issue](#).

Was this page helpful?

 Yes

 No



Resources

[Forum](#)

[Changelog](#)

[LangChain Academy](#)

[Contact Sales](#)

Company

[Home](#)

[Trust Center](#)

[Careers](#)

[Blog](#)